

Doubly Linked List – Delete From Beginning And Time Complexity.

Program :

```
void deleteFromBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    Node *temp = head;
    head = head->next;
    if (head != nullptr)
    {
        head->prev = nullptr;
    }
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
    temp->next = nullptr;
    free(temp);
    temp = nullptr;
}
... .
case 5:
    deleteFromBeginning();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by a function, excluding the space taken by the inputs themselves.

In the deleteFromBeginning function, the only new variable created is:

*Node *temp: A single temporary pointer used to hold the address of the node being deleted. The size of this pointer is constant.*

The function's main memory operation is free(temp), which deallocates memory, not allocates it. Since the amount of extra memory used does not grow with the size of the linked list, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter-only Definition

This view considers only the space taken by the function's direct parameters. The function signature is void deleteFromBeginning().

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes before the deletion, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$.

Algorithm: Delete from Beginning (Doubly Linked List)

DDELBEG(START)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete." and EXIT.

[End of If structure.]

2. [STORE ADDRESS OF FIRST NODE]

Set PTR := START [PTR is a pointer used to hold the node to be deleted].

3. [ADVANCE START TO THE SECOND NODE]

Set START := NEXTLINK[START].

4. [UPDATE NEW FIRST NODE, IF IT EXISTS]

If START ≠ NULL, then:

Set PREVLINK[START] := NULL (The PREV pointer of the new first node is set to NULL to unlink from previous first node).

[End of If structure.]

5. [OUTPUT THE DELETED NODE]

Write: "DELETED", INFO[PTR], "from the beginning."

6. [DEALLOCATE THE ORIGINAL FIRST NODE]

Set NEXTLINK[PTR] := NULL (The NEXT pointer of the previous first node de – links from the current and new first node)

free(PTR) [Releases the memory of the node pointed to by PTR].

7. [AVOID DANGLING POINTER]

Set PTR := NULL.

8. *Exit*
-